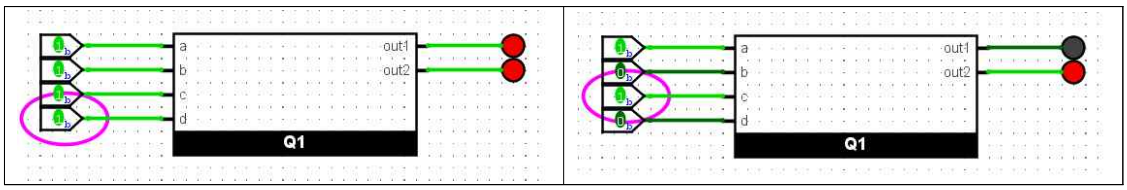
Assignment-2

학번: 2024079825
이름: 김지우

1. AND GATE & OR GATE / code



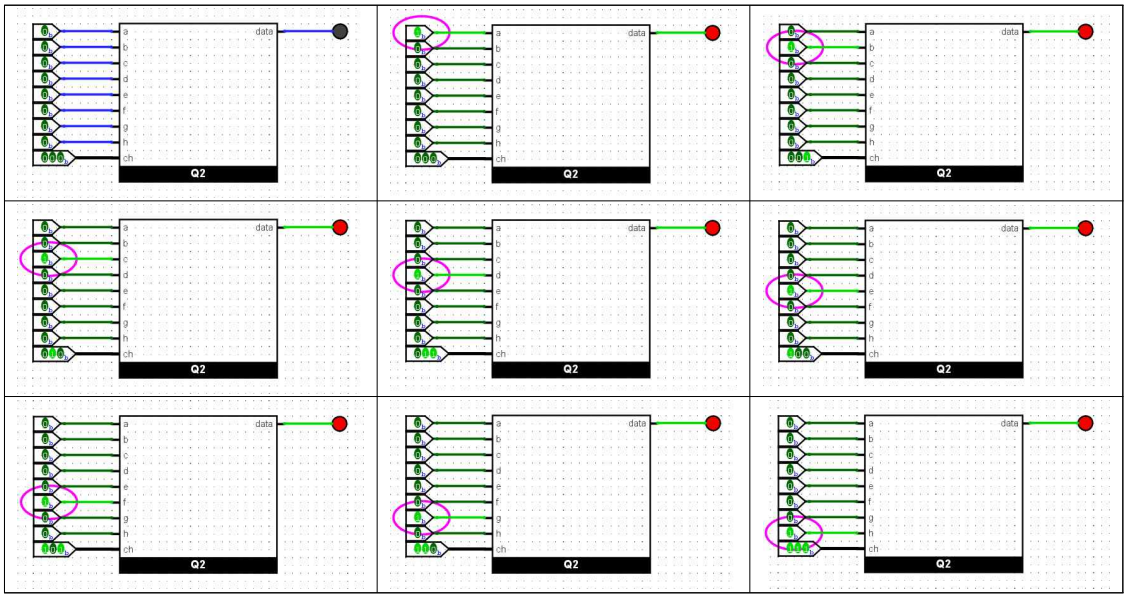

```vhdl
-- library and package
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_arith.all;
use ieee.std_logic_unsigned.all;

-- entity
entity Q1 is
    port(
          a, b, c, d: in std_logic;     -- four 1-bit input signals
          out1, out2: out std_logic     -- two 1-bit output signals
        );
end entity Q1;

-- architecture
architecture bhv of Q1 is
begin
    out1 <= a and b and c and d;
    out2 <= a or b or c or d;
end architecture bhv;
```

4개의 입력 a, b, c, d에 대해 AND 및 OR 연산을 수행하도록 구현하였다. AND 게이트는 모든 입력이 1일 때만 out1이 1이 되고, OR 게이트는 하나 이상의 입력이 1이면 out2가 1이 된다. 입력값을 변경하며 두 출력이 각 게이트의 논리 연산에 맞게 동작하는지 확인하였다.

2. 8-to-1 Multiplexer / code



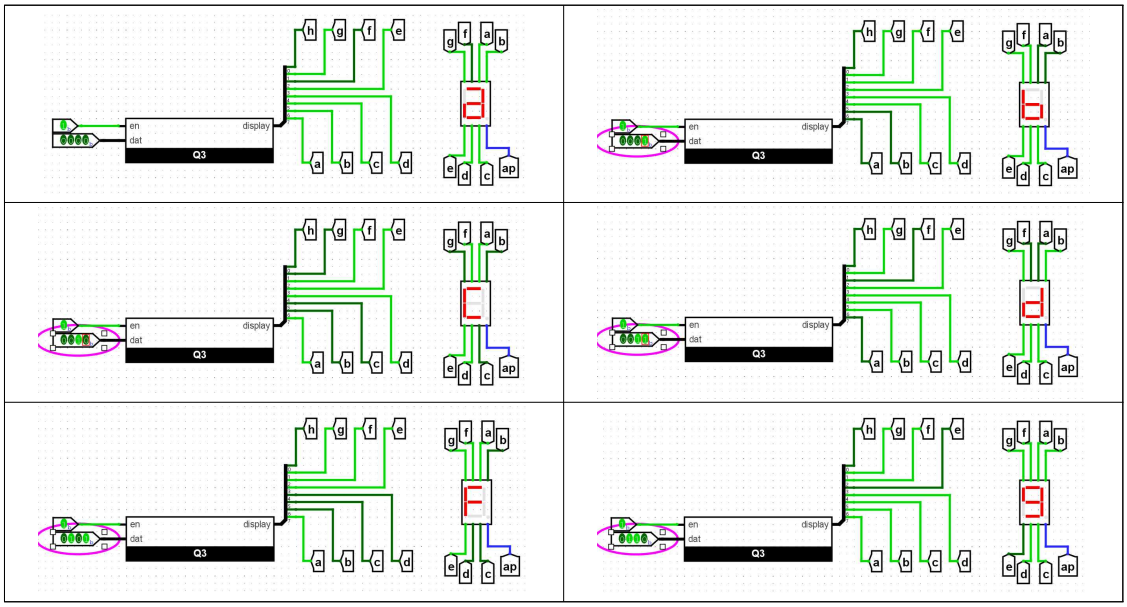

```vhdl
-- library and package
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_arith.all;
use ieee.std_logic_unsigned.all;

-- entity
entity Q2 is
    port(
          a, b, c, d, e, f, g, h: in std_logic;     -- 8 input signals
          ch: in std_logic_vector(2 downto 0);     -- 3-bit selector
          data: out std_logic     -- output signal
        );
end entity Q2;

-- architecture
architecture bhv of Q2 is
begin
    process(a, b, c, d, e, f, g, h, ch)
    begin
        case ch is
            when "000" => data <= a;     -- if ch = 0, choose input a
            when "001" => data <= b;     -- if ch = 1, choose input b
            when "010" => data <= c;     -- if ch = 2, choose input c
            when "011" => data <= d;     -- if ch = 3, choose input d
            when "100" => data <= e;     -- if ch = 4, choose input e
            when "101" => data <= f;     -- if ch = 5, choose input f
            when "110" => data <= g;     -- if ch = 6, choose input g
            when "111" => data <= h;     -- if ch = 7, choose input h
            when others => data <= '0';
        end case;
    end process;
end architecture bhv;
```

8-to-1 멀티플렉서는 8개의 입력 중 하나를 선택하여 출력하는 회로이다. 3비트 선택 신호 ch가 000부터 111까지 변함에 따라 입력 a부터 h까지 순서대로 선택되어 data로 출력된다. 선택 신호와 입력값을 변경하며 선택된 입력이 출력에 반영되는지 확인하였다.

3. 7-Segment display / code



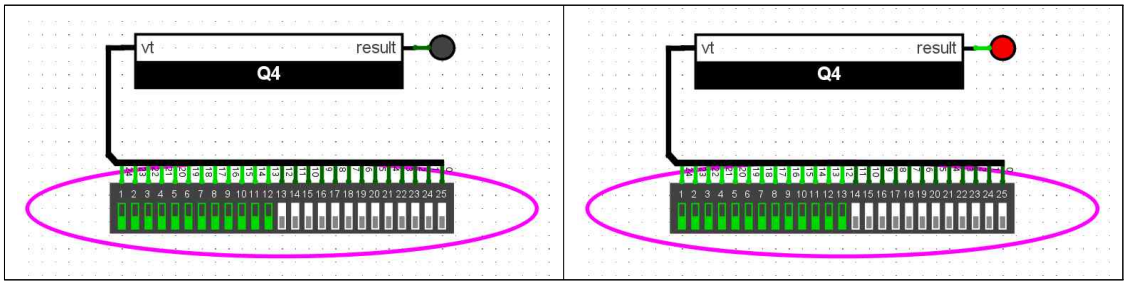

```vhdl
-- library and package
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_arith.all;
use ieee.std_logic_unsigned.all;

-- entity
entity Q3 is
    port(
          en: in std_logic;     -- enable signal (active high)
          dat: in std_logic_vector(3 downto 0);     -- 4-bit input data
          display: out std_logic_vector(7 downto 0)     -- 8-bit 7-segment output
        );
end entity Q3;

-- architecture
architecture bhv of Q3 is
begin
    process(en, dat)
    begin
        if en = '0' then
            display <= "00000000";
        else
            case dat is
                when "0000" => display <= "11111010";     -- pattern for 'a'
                when "0001" => display <= "00111110";     -- pattern for 'b'
                when "0010" => display <= "10011100";     -- pattern for 'c'
                when "0011" => display <= "01111010";     -- pattern for 'd'
                when "0101" => display <= "10001110";     -- pattern for 'f'
                when "0110" => display <= "11110110";     -- pattern for 'g'
                when others => display <= "00000000";     -- default: display OFF
            end case;
        end if;
    end process;
end architecture bhv;
```

입력 dat의 값에 따라 7세그먼트에 문자 a, b, C, d, f, g를 표시하도록 구현하였다. en이 1일 때는 입력값에 해당하는 문자를 표시하고, 0일 때는 모든 세그먼트를 끈다. 각 문자에 해당하는 입력값을 설정하여 세그먼트의 점등 형태를 확인하였다.

4. 25-Person Voting Machine / code




```vhdl
-- library and package
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_arith.all;
use ieee.std_logic_unsigned.all;

-- entity
entity Q4 is
    port(
          vt: in std_logic_vector(25 downto 1);     -- 25-bit input vector
          result: out std_logic     -- output: 1 if count >= 13
        );
end entity Q4;

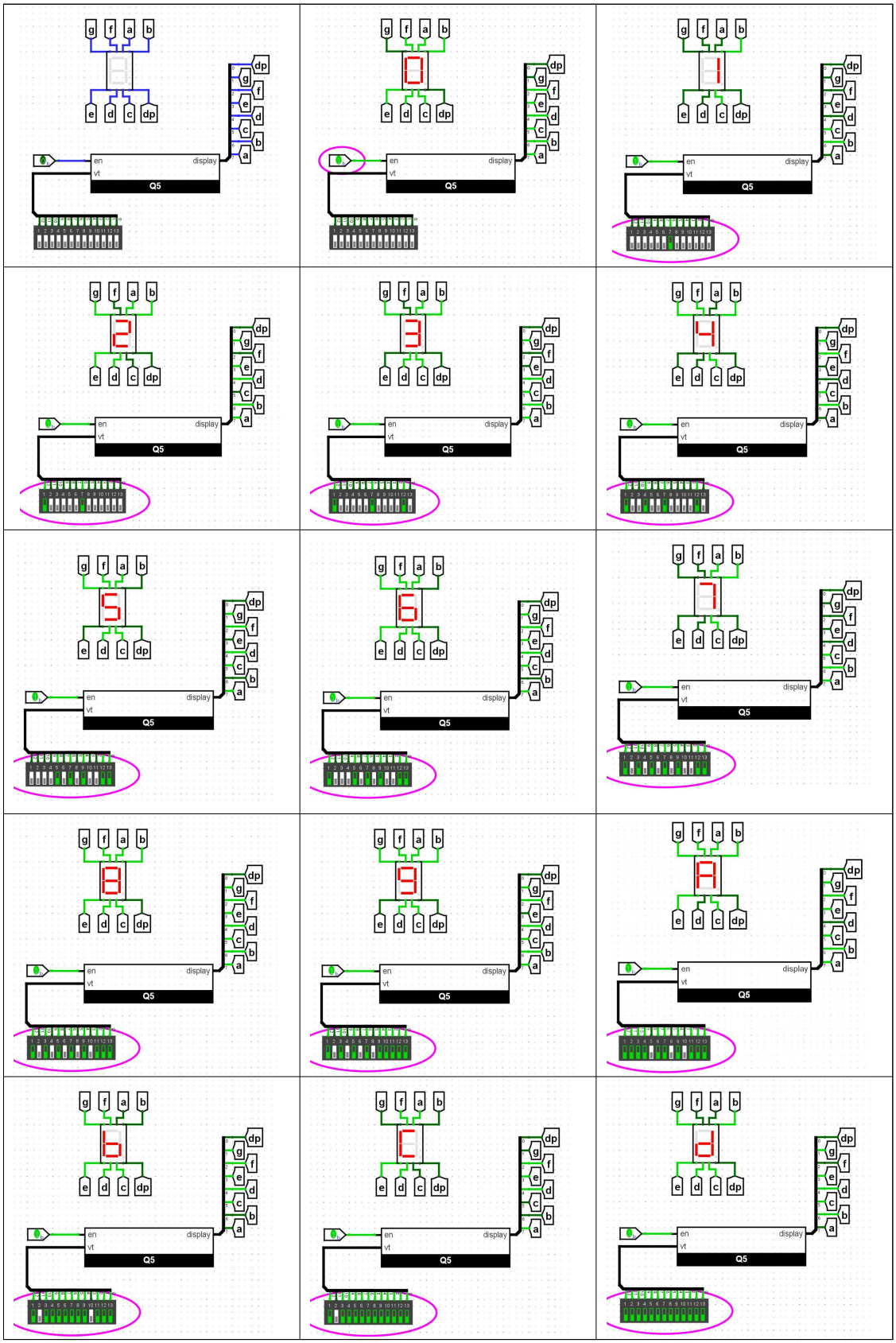
-- architecture
architecture bhv of Q4 is
begin
    process(vt)
        variable sum: integer range 0 to 25;
    begin
        sum := 0;

        for i in 1 to 25 loop
            if vt(i) = '1' then
                sum := sum + 1;
            end if;
        end loop;

        if sum >= 13 then
            result <= '1';
        else
            result <= '0';
        end if;
    end process;
end architecture bhv;
```

25인 투표기는 입력 vt에서 찬성을 의미하는 1의 개수를 세어 과반수 여부를 판단하는 회로이다. 찬성 입력이 12개일 때 result가 0이 되고, 13개일 때 1이 되는 것을 확인하였다. 이를 통해 전체 25명 중 13명 이상이 찬성하면 가결되도록 구현하였음을 확인하였다.

5. Counting 1s & 7-Segment Display / code



```vhdl
-- library and package
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_arith.all;
use ieee.std_logic_unsigned.all;

-- entity
entity Q5 is
    port(
          en: in std_logic;     -- enable signal
          vt: in std_logic_vector(12 downto 0);     -- 13-bit input vector
          display: out std_logic_vector(7 downto 0)     -- 7-segment output pattern
        );
end entity Q5;

-- architecture
architecture bhv of Q5 is
begin
    process(en, vt)
    variable sum: integer range 0 to 15;
    begin
        sum:= 0;
        if en = '0' then
            display <= "00000000";
        else
            for i in 0 to 12 loop
                if (vt(i) = '1') then
                    sum:= sum + 1;
                end if;
            end loop;
            case sum is
                when 0 => display <= "11111100";     -- pattern for '0'
                when 1 => display <= "01100000";     -- pattern for '1'
                when 2 => display <= "11011010";     -- pattern for '2'
                when 3 => display <= "11110010";     -- pattern for '3'
                when 4 => display <= "01100110";     -- pattern for '4'
                when 5 => display <= "10110110";     -- pattern for '5'
                when 6 => display <= "10111110";     -- pattern for '6'
                when 7 => display <= "11100000";     -- pattern for '7'
                when 8 => display <= "11111110";     -- pattern for '8'
                when 9 => display <= "11110110";     -- pattern for '9'
                when 10 => display <= "11101110";     -- pattern for 'A'
                when 11 => display <= "00111110";     -- pattern for 'B'
                when 12 => display <= "10011100";     -- pattern for 'C'
                when 13 => display <= "01111010";     -- pattern for 'D'
                when 14 => display <= "10011110";     -- pattern for 'E'
                when 15 => display <= "10001110";     -- pattern for 'F'
                when others => display <= "00000000";     -- default: display OFF
            end case;
        end if;
    end process;
end architecture bhv;
```

13비트 입력 vt에 포함된 1의 개수를 세어 7세그먼트에 표시하도록 구현하였다. 1의 개수가 0~9개이면 해당 숫자를, 10~13개이면 각각 A, b, C, d를 표시한다. 또한 en이 0이면 표시가 꺼지고, 1이면 계산 결과가 표시되도록 구현하였다. 코드에는 E와 F의 표시 패턴도 포함하였으나, 13비트 입력에서는 1의 개수가 최대 13개이므로 해당 값은 발생하지 않는다.